

Smart Asset Management and Resource Allocation Platform

Design Document

Problem Understanding

Organizations such as the Cultural Council of IIT Roorkee manage cameras, lighting, audio gear, costumes, stage props and event infrastructure across many teams and events. Fragmented tracking causes unclear availability, booking conflicts, late returns and weak accountability. The platform centralizes inventory, booking requests, approval decisions, issue and return tracking, borrowing history and analytics.

System Architecture

The application uses a browser frontend, a Python REST-style API server, a SQLite database and signed HTTP-only cookie sessions. The frontend is served by the same backend to keep local setup simple.

Layer	Responsibility	Technology
Frontend	Single-page user interface, role-aware controls, charts and forms	HTML, CSS, JavaScript
Backend	API routes, authentication, authorization, booking workflow	Python HTTP server
Database	Users, assets, bookings and lifecycle state	SQLite
Security	Password hashing and signed sessions	PBKDF2, HTTP-only cookies

Database Schema

Entity	Important Fields	Purpose
users	id, name, email, password_hash, role	Stores identity and role-based access
assets	id, asset_name, category, description, quantity_total, status	Stores inventory and available stock base
bookings	id, asset_id, user_id, quantity_requested, start_date, end_date, status	Stores request, approval, issue, return and overdue lifecycle

Entity Relationship Diagram

Relationship	Cardinality	Meaning
users -> bookings	1 to many	A user can create many booking requests
assets -> bookings	1 to many	An asset can appear in many booking records
bookings	many to 1 user and asset	Each request connects one user with one asset

API Overview

Bucket	Routes	Purpose
Authentication	POST /api/register, POST /api/login, POST /api/logout, GET /api/me	Account access and session handling
Inventory	GET /api/assets, POST /api/assets, PUT /api/assets/:id, DELETE /api/assets/:id	Discover assets, admin inventory management
Bookings	GET /api/bookings, POST /api/bookings, POST /api/bookings/:id	Request and allocation, lifecycle
Analytics	GET /api/analytics	Summary cards, utilization and status mix

Design Decisions

The stack is intentionally lightweight and reproducible. Python standard library APIs avoid dependency installation, SQLite keeps data local and portable, and vanilla JavaScript keeps the frontend readable. Backend route checks enforce admin-only actions, while frontend controls adapt to the current role.

Inventory integrity is maintained by calculating availability from approved, issued and overdue bookings that overlap the requested date range. Source-of-truth quantity remains in the assets table, while booking records preserve the full lifecycle from pending request to return.

Mandatory Deliverables Mapping

Deliverable	Included Output
Working application	Source code in asset-management-platform
Design document PDF	smart-asset-management-design-document.pdf
Public GitHub repository contents	Source, config, database scripts and docs are organized for upload
README.md	Project overview, stack, setup, running guide and feature list
Demonstration video	README includes a 3 minute workflow checklist for recording

Scalability Considerations

The schema can grow into event, section, notification, audit log or maintenance modules. API boundaries also make it straightforward to move from SQLite to PostgreSQL if concurrent usage increases.